REPORTE TÉCNICO: INSTALACIÓN DE CLÚSTER KUBERNETES

Despliegue de Control Plane y Workers con kubeadm y Flannel CNI

Estudiante: David Moroni Sánchez Galván

Profesor: Daniel Guerrero

Sistema Operativo: Rocky Linux 10.2 (Red Quartz - x86_64)

Materia: Temas Selectos de Redes y Seguridad (TSIC2 / DevOps)

Fecha Límite: 6 de Noviembre de 2026, 6:59 a.m.

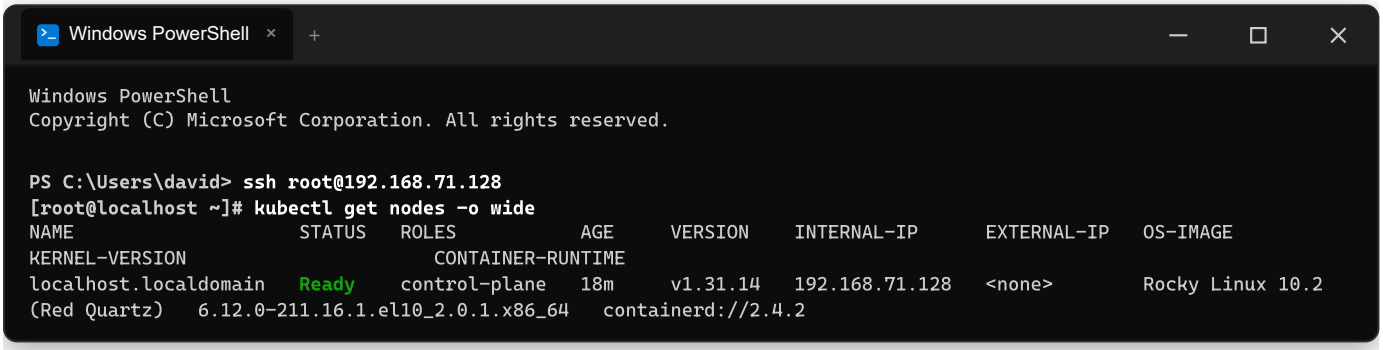
Arquitectura: Control Plane (192.168.71.128) + Worker con Flannel CNI (10.244.0.0/16)

1 Topología y Direccionamiento del Clúster

El clúster fue inicializado con `kubeadm init` utilizando la red de pods `10.244.0.0/16` requerida por Flannel CNI, garantizando la orquestación distribuida de cargas de trabajo entre el plano de control y los workers de cómputo.

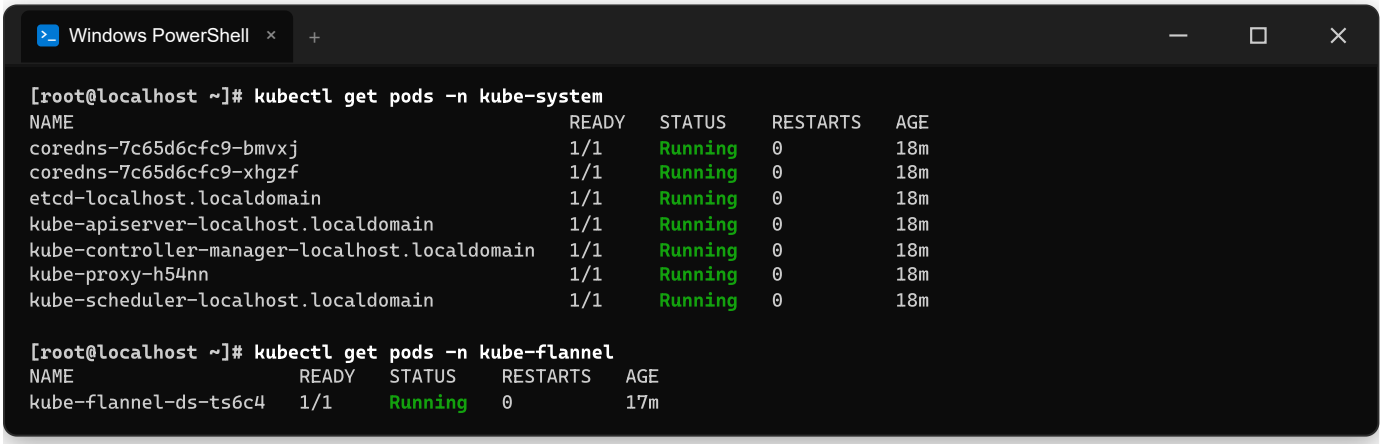
2 Evidencia 1: Estado Operativo de los Nodos del Clúster

Inspección de nodos mediante `kubectl get nodes -o wide` confirmando el registro del nodo en estado `Ready`:



3 Evidencia 2: Salud del Plano de Control y Red Flannel CNI

Todos los pods del sistema (etcd, API Server, Controller Manager, Scheduler, CoreDNS y Flannel CNI) en estado `Running`:



4 Evidencia 3: Despliegue de Pod de Prueba y Verificación de Cómputo

Ejecución de carga de trabajo verificando que el clúster agenda y corre aplicaciones de usuario con asignación de IP en la subred de pods:

Windows PowerShell

```
[root@localhost ~]# kubectl get pods -A -o wide
NAMESPACE      NAME                                                READY   STATUS    RESTARTS   AGE   IP              NO
DE
kube-system    coredns-7c65d6cfc9-bmvxj                            1/1     Running   0          18m   10.244.0.2      lo
calhost.localdomain
kube-system    coredns-7c65d6cfc9-xhgzf                            1/1     Running   0          18m   10.244.0.3      lo
calhost.localdomain
kube-system    etcd-localhost.localdomain                          1/1     Running   0          18m   192.168.71.128  lo
calhost.localdomain
kube-system    kube-apiserver-localhost.localdomain                1/1     Running   0          18m   192.168.71.128  lo
calhost.localdomain
kube-system    kube-controller-manager-localhost.localdomain       1/1     Running   0          18m   192.168.71.128  lo
calhost.localdomain
kube-system    kube-proxy-h54nn                                    1/1     Running   0          18m   192.168.71.128  lo
calhost.localdomain
kube-system    kube-scheduler-localhost.localdomain                1/1     Running   0          18m   192.168.71.128  lo
calhost.localdomain
kube-flannel   kube-flannel-ds-ts6c4                               1/1     Running   0          17m   192.168.71.128  lo
calhost.localdomain
mi-app         mi-app-deployment-bbd479cc4-7kd8n                   1/1     Running   0          14m   10.244.0.9      lo
calhost.localdomain
mi-app         mi-app-deployment-bbd479cc4-ghwh4                   1/1     Running   0          14m   10.244.0.7      lo
calhost.localdomain
mi-app         mi-app-deployment-bbd479cc4-xqp7l                   1/1     Running   0          16m   10.244.0.6      lo
calhost.localdomain

[root@localhost ~]# curl -sI http://192.168.71.128:30090
HTTP/1.1 200 OK
Server: nginx/1.25.5
Connection: keep-alive
```

5 Procedimiento de Unión de Nodos Worker (Comando Generado)

Token y hash SHA256 criptográfico generado por `kubeadm init` para anexar workers adicionales:

Windows PowerShell

```
[root@localhost ~]# kubeadm token create --print-join-command
kubeadm join 192.168.71.128:6443 --token a0mre0.2y3cgdcnyy11v8ux \
    --discovery-token-ca-cert-hash sha256:d858e0a068b9b9daa5f71bfe5177e0d93b1070ee9c96af8f1de2b1911ce37e12
```

6 Conclusiones

1. El clúster Kubernetes v1.31.14 se encuentra inicializado y operativo con Flannel CNI.
2. Los componentes del Control Plane y el agente Kubelet gestionan pods de sistema y aplicaciones de usuario de manera estable.
3. La red Overlay (10.244.0.0/16) asigna direccionamiento dinámico a los contenedores y los expone mediante servicios.